What is Flask?

The Micro-Framework for Python

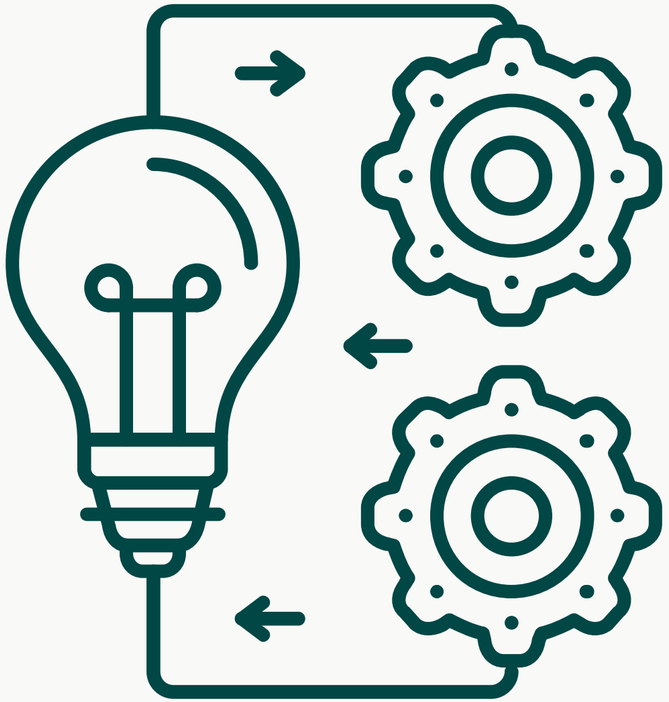
Flask is a "micro" web framework written in Python, meaning it provides the essential tools to build a website without forcing a specific project structure. Unlike heavier frameworks, it is lightweight and flexible, allowing developers to add only the libraries they actually need. It is built on the Werkzeug WSGI toolkit and the Jinja2 template engine, making it a favorite for beginners and professionals alike.

Flask

Core Components of Flask

Routing, Templates, and Development

The heart of Flask lies in Routing, which maps specific URLs (like /home) to Python functions that handle the request. It uses Jinja2 Templates to separate the Python logic from the HTML presentation, allowing for dynamic content rendering. Additionally, Flask comes with a built-in development server and debugger, which helps students see their changes instantly during the learning process.



The "Hello World" Code

The "Hello World" Code

To create a basic application, you only need five lines of code: you import the Flask class, create an instance, define a route using a decorator, and write a function that returns a string. This minimalist approach allows students to focus on the logic of the web request rather than boilerplate code. Running this script starts a local server where the "Hello World" message appears in the browser.

requirements.txt | hello.py

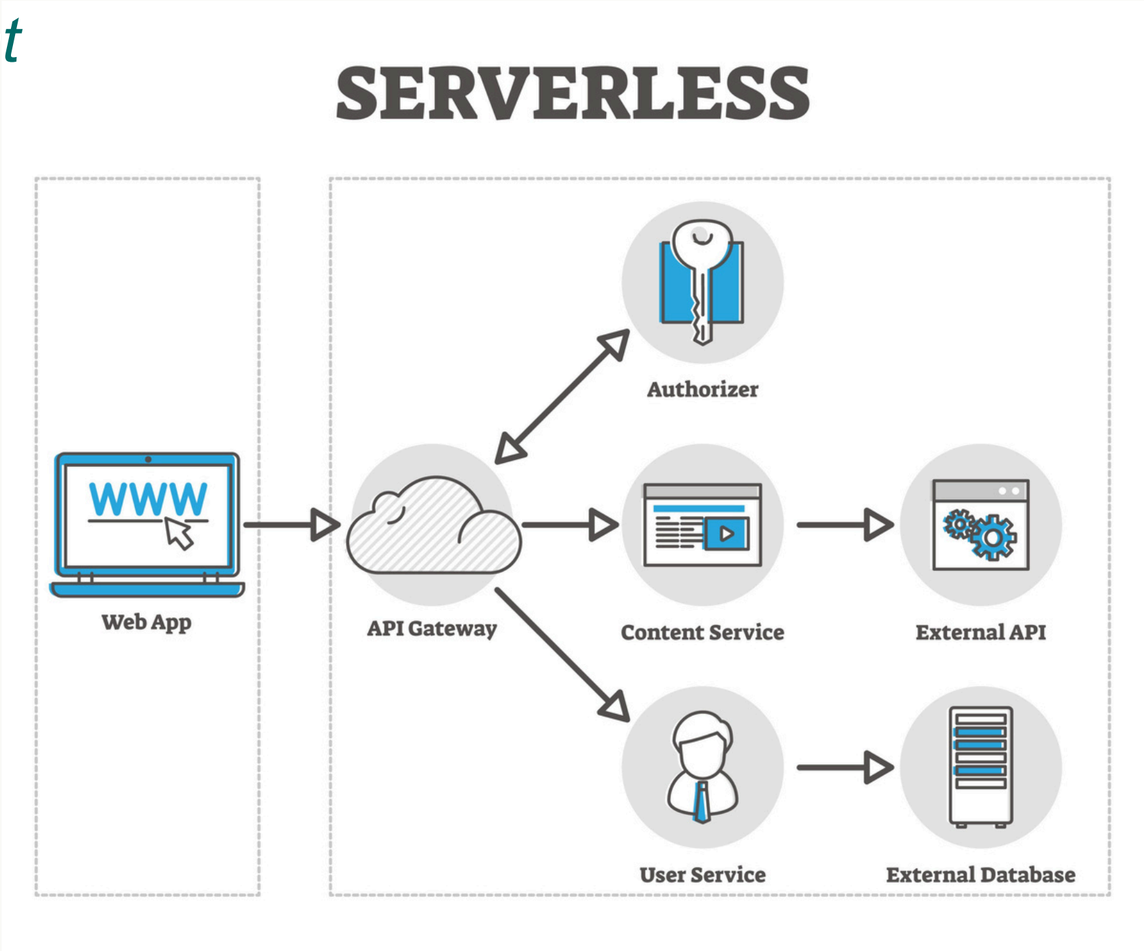
```python
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello World!"
```

Understanding "Serverless"

Code Execution Without Server Management

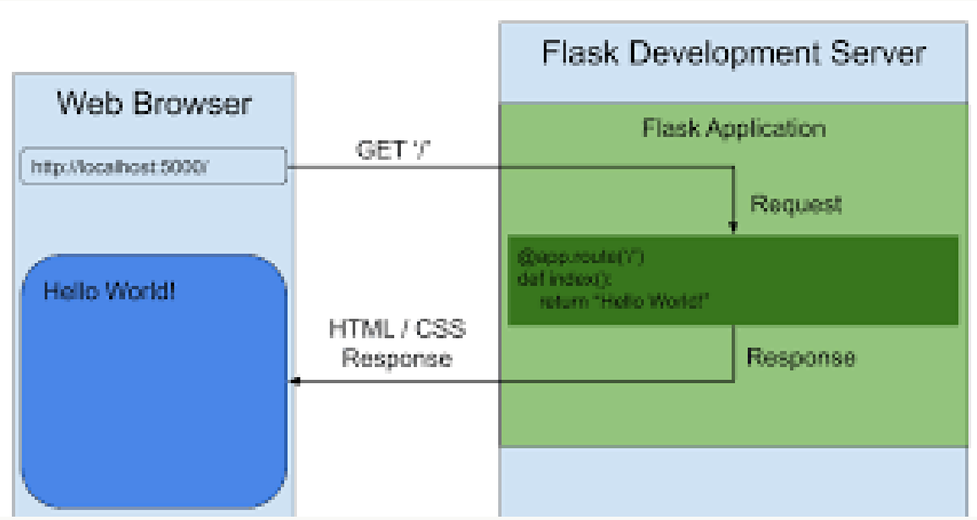
Serverless doesn't mean there are no servers; it means the developer doesn't have to manage, patch, or scale them. In a serverless model, the cloud provider (like AWS, Google Cloud, or Vercel) automatically handles the infrastructure, starting the environment only when a request comes in. This is highly cost-effective because you only pay for the exact duration the code is running, rather than paying for an idle server.



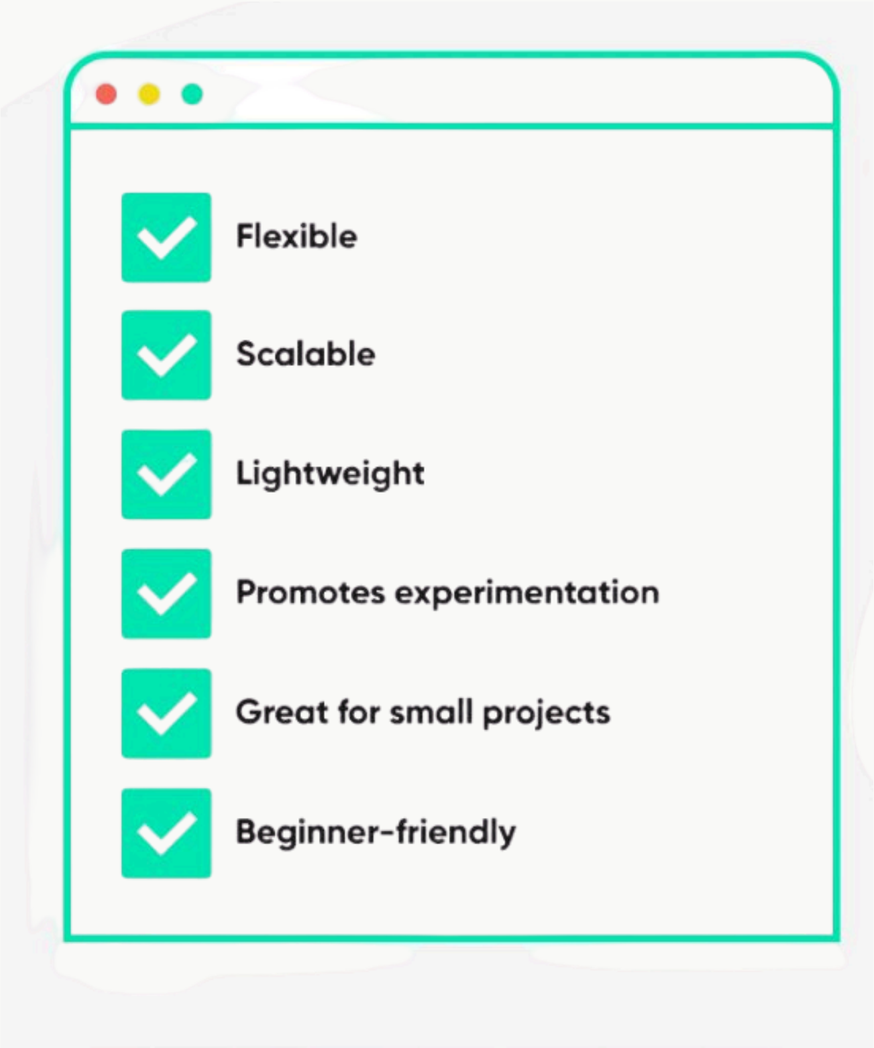
How Flask Works in Serverless

From Persistent Servers to On-Demand Functions

In a traditional setup, a Flask app runs 24/7 on a virtual machine waiting for traffic. In a serverless environment, the Flask app is wrapped into a single "function" that is triggered by an HTTP request via an API Gateway. When the request hits the gateway, the cloud provider spins up a tiny container, runs your Flask code to generate a response, and then shuts it down immediately after.



Best Practices for Students

Keeping Functions Small and Fast

When teaching serverless, emphasise that "Cold Starts"—the delay when a function starts for the first time—are a key consideration. To keep apps fast, students should keep their Flask applications modular and avoid importing heavy libraries that aren't strictly necessary. Using specialised tools like Zappa (for AWS) or Vercel's Python Runtime makes deploying a Flask app to the cloud a seamless, one-command process.